



Deployment architectures

Architecturally, you can deploy these models in a few distinct ways:

- Running directly on a public cloud GPU instance with a standard operating system.
- Containerising the open source LLM into a Docker image and deploying it within a Kubernetes cluster.
- Consuming the models directly through managed API endpoints.

Open source LLMs and their use cases

Speech-to-text: Models like Whisper Large V3 provide reliable audio transcription capabilities comparable to proprietary models.

Vision-language models (VLMs) and object detection: By using open source models like Qwen VLM, you can pass an image, process the visual features, and request structured outputs like JSON formatting to categorise elements, people, or landscape components within the frame.

RAG systems and content generation pipelines: You can construct complete retrieval-augmented generation (RAG) pipelines using strictly open source components. For instance, combining a vector database for context retrieval with an Ollama-hosted instruct model yields accurate, structured responses. This RAG core

can be extended into a complex multi-model pipeline: the generated content is formatted into structured JSON, an open source text-to-speech model such as NVIDIA TTS model Nvr-tts-en-us converts that JSON into narration voice files, and a script built on python-pptx generates the matching slide deck — with tools like FFmpeg finally combining the audio and visual outputs into full video content suitable for enterprise training or knowledge bases.

Maintaining output quality in these systems relies on the underlying dataset. Your knowledge base must prioritise data quality and proper cleaning, such as structured processing of technical documentation across domain-specific topics, rather than simply relying on sheer volume before vector embedding.

Another practical use case is interacting with models directly from your terminal. Open source utilities like SHAI allow you to SSH into an AI system. By linking this CLI tool to an open-weight model like DeepSeek R1, you can use standard natural language to trigger workflows or deploy sub-agents right from your local machine, paying only for the API tokens consumed.

Similarly, developer tools like kilocode can connect to open source models, such as gpt-oss, for code generation, giving you a viable alternative to proprietary coding assistants.

Production considerations: Security, governance and fine tuning

When moving these setups into production, two operational areas are critical:

- **Data governance and guardrails:** Hosting models yourself, whether on-premises, via private cloud, or via public cloud, keeps your data strictly in-house. To manage this safely, you need to implement guardrails that filter content and enforce internal policies both before a prompt reaches the model and after it generates a response.
- **Infrastructure security:** Standard self-hosting rules apply here: you need strict firewall configurations and port management. If you are deploying via Kubernetes, that security footprint must expand to include container image scanning and strict network policies to control the traffic moving between your containers.

Fine tuning large models carries high computational costs and should be carefully evaluated against your requirements. For most standard enterprise use cases, an RAG architecture provides the necessary domain context without the need to retrain weights. Fine tuning is typically reserved for highly specialised domains such as medical imaging. When fine tuning is necessary, smaller open source models, typically under 8 to 10 billion parameters, combined with parameter-efficient methods like LoRA or QLoRA, optimise resource utilisation. **END** 

This article is based on the talk at AI DevCon India 2026 titled 'Practical Applications of Open Source LLMs', delivered by Akki Bharath Goud, cloud architect, OVHcloud. It has been transcribed and curated by Jiya Jay Singh, journalist at the EFY Group.